Modelado algorítmico de la generación hidroeléctrica del Sistema Interconectado Nacional durante el Fenómeno de El Niño: Fase 2

[Nombre completo del estudiante 1]
[Nombre completo del estudiante 2]
[Nombre completo del estudiante 3]
Departamento de Energía, Universidad de la Costa (CUC)
Programa de Ingeniería Eléctrica
[Código y nombre de la asignatura]
[Nombre del docente]
16 de octubre de 2026

Fase 2: Desarrollo progresivo con condicionales y ciclos

Contexto y alcance

El sistema eléctrico colombiano es predominantemente hidroeléctrico y su confiabilidad disminuye durante el Fenómeno de El Niño (Restrepo-Trujillo et al., 2022), que reduce los caudales de la región andina (Poveda et al., 2001; Gómez-Dueñas et al., 2024) y pone a prueba la energía firme del sistema (Cramton y Stoft, 2007; Moreno Chuquen y Cantillo Luna, 2020). En la Fase 1 se diseñó un algoritmo que, con datos básicos de un grupo de centrales, calcula la energía generable del mes con la ecuación de potencia hidráulica $P = \rho \cdot g \cdot Q \cdot H \cdot \eta$ (Sritram y Suntivarakorn, 2017), hace el balance del embalse y clasifica el riesgo de desabastecimiento.

En esta Fase 2 el algoritmo se implementa en Python como un programa con menú que permite registrar centrales, consultarlas y generar el reporte del subsistema, usando condicionales y ciclos. Además, clasifica el estado de cada central para identificar instalaciones deficientes (vertimiento, subutilización o eficiencia baja), lo que responde al eje problematizador y permite estimar cuándo es necesario el respaldo térmico (Medina y Moreno, 2007). Quedan por fuera el despacho económico y el tratamiento estocástico de los caudales (Hoseinzadeh et al., 2020; Tayyeh y Mohammed, 2024) y el almacenamiento de datos en archivos.

Atención a las observaciones de la Fase 1

Tabla 1

Observaciones de la Fase 1 y ajustes realizados

Observación	Ajuste realizado
Referencias CREG y XM no provenían de bases de datos institucionales.	Se reemplazaron por artículos de IEEE Xplore y Scopus (Cramton y Stoft, 2007; Restrepo-Trujillo et al., 2022).
La autonomía aumentaba en escenarios más secos.	Se calcula con el turbinado necesario para cubrir la demanda AUT = V_f / [(V_t/d)·(D_R/E_T)].
No se probó la validación por central.	Casos V-09 a V-11 de la Tabla 8.
Horas máximas fijas (744 h) y volumen inicial sin tope.	Reglas RN-02 y RN-03.
[Observación de la docente pendiente por transcribir]	[Ajuste]

Nota. Elaboración propia.

Variables y restricciones

Tabla 2

Variables principales del programa

Variable	Tipo Python (PSeInt)	Restricción	Ámbito
GRAVEDAD, DENSIDAD, U_*	float (Real)	Constantes	Global, solo lectura
opcion	int (Entero)	1 a 5	Programa principal
generales	dict (varias variables)	días 28–31; horas 1 a 24·días; 0 ≤ vi ≤ vu	Programa principal

Variable	Tipo Python (PSeInt)	Restricción	Ámbito
centrales	list de dict (arreglos[50])	0 a 50 centrales, nombre único	Programa principal
qa, qt, h, ef, pn	float (Real)	qa ≥ 0; qt, h, pn > 0; 0 < ef ≤ 1	Elemento de la lista
pe, e, fp	float (Real)	≥ 0	Local de calcular_central
estado, riesgo, alerta	str (Caracter)	4, 4 y 3 valores posibles	Locales
et, va, vt, pinst, malas	float / int	acumuladores que inician en 0	Local de generar_reporte
nino	bool (Lógico)	True si IA < 0,80	Local de generar_reporte

Nota. Elaboración propia.

Tabla IPO

Tabla 3

Tabla IPO por opción del menú

Opción	Entrada	Proceso	Salida
1. Datos generales	Región, días, horas, volúmenes, caudal medio, demanda	Validar rangos (RN-02, RN-03)	Datos generales
2. Registrar	Nombre, Q_af, Q_t, H, η, P_n	Validar; rechazar repetidos con for (RN-01, RN-06)	Central agregada
3. Consultar	Nombre	Buscar con for; calcular P_e, E, FP y estado	Datos de la central
4. Reporte	Lista de centrales y datos generales	Acumular con for; balance; indicadores; riesgo y alerta	Reporte del subsistema
5. Salir	—	Terminar el ciclo while	Fin

Nota. Elaboración propia.

Reglas de negocio

Tabla 4

Reglas de negocio

Código	Regla	Estructura en Python
RN-01	Máximo 50 centrales.	if len(centrales) >= 50
RN-02	Días de 28 a 31; horas de 1 a 24 × días.	leer_numero con límites
RN-03	Volumen útil > 0 y 0 ≤ volumen inicial ≤ volumen útil.	leer_numero con límites
RN-04	La potencia efectiva no supera la nominal.	if / else
RN-05	Estado de la central: LIMITADA, SUBUTILIZADA (FP < 30 %), EFICIENCIA BAJA (η < 0,85) o ADECUADA.	if / elif / else
RN-06	No se repiten nombres de central.	for con if
RN-07	Riesgo: NORMAL (nivel > 70 % y cobertura ≥ 100 %), SEGUIMIENTO (> 50 %), ALERTA (> 30 %), CRÍTICO.	if / elif / else
RN-08	Alerta: ACTIVAR RESPALDO si CRÍTICO o déficit con El Niño; VIGILANCIA si solo uno; si no, NORMAL.	and / or
RN-09	No se consulta ni reporta sin datos generales y al menos una central.	if anidado
RN-10	Hay condición El Niño si el índice de aportes es menor que 0,80.	expresión lógica

Nota. Los umbrales de RN-05 son valores didácticos definidos por el grupo. Elaboración propia.

Condicionales y ciclos utilizados

Las mayores dificultades de quienes inician en programación están en el uso conceptual de condicionales y ciclos (Qian y Lehman, 2017) y su dominio es parte del pensamiento computacional aplicado al diseño en ingeniería (Ehsan et al., 2020); por eso el programa usa solo

las estructuras básicas de la Tabla 5. Se eligió Python por su sintaxis clara, que permite concentrarse en el algoritmo (Fangohr, 2004; Kunkle y Allen, 2016).

Tabla 5
Estructuras de control del programa

Estructura	Python (PSeInt)	Uso en el programa
Condicional simple	if (Si)	Detectar central repetida
Condicional doble	if-else (Si-SiNo)	Limitar potencia; calcular déficit
Condicional múltiple	if-elif-else (Segun / Si anidado)	Menú, estado de la central, riesgo
Condicional compuesta	and, or, not (Y, O, NO)	Alerta operativa; RN-09
Ciclo while	while (Mientras)	Repetir el menú y repetir cada pregunta hasta un dato válido
Ciclo for	for (Para)	Registrar (buscar repetidos), consultar y generar el reporte

Nota. Elaboración propia.

Matriz de decisiones

Tabla 6
Matriz de decisiones del riesgo y del estado de la central

Condiciones	Resultado	Caso de prueba
Nivel > 70 % y cobertura ≥ 100 %	NORMAL	Esc. 1
Nivel > 50 % (sin cumplir lo anterior)	SEGUIMIENTO	Esc. 2 y 6
30 % < nivel ≤ 50 %	ALERTA	Esc. 3
Nivel ≤ 30 %	CRÍTICO	Esc. 4
P_b > P_n	LIMITADA	CH-1 y CH-2, Esc. 1
P_b ≤ P_n y FP < 30 %	SUBUTILIZADA	Esc. 4
P_b ≤ P_n, FP ≥ 30 % y η < 0,85	EFICIENCIA BAJA	CH-3, Esc. 6
Ninguna de las anteriores	ADECUADA	Esc. 2

Nota. Elaboración propia.

Tabla 7
Matriz de decisiones de la alerta operativa

¿CRÍTICO?	¿Déficit?	¿El Niño?	Alerta	Caso
No	No	No	OPERACIÓN NORMAL	Esc. 1
No	No / Sí	Sí / No	VIGILANCIA	Esc. 6 (déficit sin El Niño)
No	Sí	Sí	ACTIVAR RESPALDO	Esc. 2 y 3
Sí	Cualquiera	Cualquiera	ACTIVAR RESPALDO	Esc. 4

Nota. Elaboración propia.

Pseudocódigo

El pseudocódigo se escribió antes del programa porque facilita organizar la secuencia de pasos antes de codificar (Glorfeld y Palko, 1988). Agrega a la Fase 1 el menú con Mientras y Segun, los arreglos de centrales y la función EstadoCentral.

Figura 1

Pseudocódigo en PSeInt

```
Algoritmo HidroelectricaNino
  Definir opcion, n, i, dias, horas Como Entero
  Definir vu, vi, qmed, dem Como Real
  Definir hayGen, repetida Como Logico
  Definir nomNuevo Como Caracter
  Dimension nom[50], qa[50], qt[50], h[50], ef[50],
pn[50]
  Definir nom Como Caracter
  Definir qa, qt, h, ef, pn Como Real
  n <- 0
  hayGen <- Falso
  opcion <- 0
  Mientras opcion <> 5 Hacer
    Escribir "1.Generales 2.Registrar 3.Consultar
4.Reporte 5.Salir"
    Leer opcion
    Segun opcion Hacer
      1:
        Repetir
          Leer dias, horas, vu, vi, qmed, dem
        Hasta Que dias>=28 Y dias<=31 Y horas>=1 Y
horas<=24*dias Y vu>0 Y vi>=0 Y vi<=vu Y qmed>0 Y
dem>0
        hayGen <- Verdadero
      2:
        Si n >= 50 Entonces
          Escribir "ERROR: maximo de centrales"
        SiNo
          Leer nomNuevo
          repetida <- Falso
          Para i <- 1 Hasta n Hacer
            Si nom[i] = nomNuevo Entonces
              repetida <- Verdadero
            FinSi
          FinPara
          Si repetida Entonces
            Escribir "ERROR: central repetida"
          SiNo
            n <- n + 1
            nom[n] <- nomNuevo
            Repetir
              Leer qa[n], qt[n], h[n], ef[n], pn[n]
            Hasta Que qa[n]>=0 Y qt[n]>0 Y h[n]>0 Y
ef[n]>0 Y ef[n]<=1 Y pn[n]>0
          FinSi
        FinSi
      3, 4:
        Si NO hayGen O n = 0 Entonces
          Escribir "Registre datos generales y una
central"
        SiNo
          Si opcion = 3 Entonces
            ConsultarCentral(n, nom, qt, h, ef, pn,
dias, horas)
          SiNo
            GenerarReporte(n, nom, qa, qt, h, ef,
pn, dias, horas, vu, vi, qmed, dem)
          FinSi
        FinSi
      5:
        Escribir "Fin del programa"
      De Otro Modo:
        Escribir "ERROR: opcion no valida"
    FinSegun
  FinMientras
FinAlgoritmo

SubProceso GenerarReporte(n, nom, qa, qt, h, ef, pn,
dias, horas, vu, vi, qmed, dem)
  Definir i Como Entero
  Definir pb, pe, e, et, va, vt, qat, pinst, vf,
nivel, ia, cob, deficit, aut Como Real
  Definir riesgo, alerta Como Caracter
  et <- 0; va <- 0; vt <- 0; qat <- 0; pinst <- 0
  Para i <- 1 Hasta n Hacer
    FinSi
    e <- pe * horas / 1000
    Escribir nom[i], " ", e, " GWh ",
EstadoCentral(pb, pn[i], e, ef[i], dias)
    et <- et + e
    pinst <- pinst + pn[i]
    qat <- qat + qa[i]
    va <- va + qa[i] * 86400 * dias / 1000000
    vt <- vt + qt[i] * 3600 * horas / 1000000
  FinPara
  vf <- vi + va - vt
  Si vf > vu Entonces
    vf <- vu
  SiNo
    Si vf < 0 Entonces
      vf <- 0
    FinSi
  FinSi
  nivel <- vf / vu * 100
  ia <- qat / qmed
  cob <- et / dem * 100
  Si et < dem Entonces
    deficit <- dem - et
  SiNo
    deficit <- 0
  FinSi
  aut <- vf / ((vt / dias) * (dem / et))
  Si nivel > 70 Y cob >= 100 Entonces
    riesgo <- "NORMAL"
  SiNo
    Si nivel > 50 Entonces
      riesgo <- "SEGUIMIENTO"
    SiNo
      Si nivel > 30 Entonces
        riesgo <- "ALERTA"
      SiNo
        riesgo <- "CRITICO"
      FinSi
    FinSi
  FinSi
  Si riesgo = "CRITICO" O (deficit > 0 Y ia < 0.8)
Entonces
    alerta <- "ACTIVAR RESPALDO TERMICO"
  SiNo
    Si deficit > 0 O ia < 0.8 Entonces
      alerta <- "VIGILANCIA"
    SiNo
      alerta <- "OPERACION NORMAL"
    FinSi
  FinSi
  Escribir et, nivel, ia, cob, deficit, aut, riesgo,
alerta
FinSubProceso

SubProceso estado <- EstadoCentral(pb, pn, e, ef,
dias)
  Definir estado Como Caracter
  Definir fp Como Real
  fp <- e / (pn * dias * 24 / 1000) * 100
  Si pb > pn Entonces
    estado <- "LIMITADA"
  SiNo
    Si fp < 30 Entonces
      estado <- "SUBUTILIZADA"
    SiNo
      Si ef < 0.85 Entonces
        estado <- "EFICIENCIA BAJA"
      SiNo
        estado <- "ADECUADA"
      FinSi
    FinSi
  FinSi
FinSubProceso
```

```
    pb <- 1000 * 9.81 * qt[i] * h[i] * ef[i] /
1000000
    Si pb > pn[i] Entonces
      pe <- pn[i]
    SiNo
      pe <- pb
```

Nota. ConsultarCentral aplica a una sola central el mismo cálculo del ciclo Para de GenerarReporte. Elaboración propia.

Programa en Python

El programa se organiza en funciones de lectura con validación, una función de cálculo por central, una función por cada opción y un menú principal controlado por un ciclo while.

Figura 2

Programa en Python (generacion_hidro_nino.py)

```
# ABP Fase 2 - Generacion hidroelectrica durante El Nino
GRAVEDAD, DENSIDAD = 9.81, 1000
U_NORMAL, U_SEGUIM, U_ALERTA = 70, 50, 30    # % del volumen util
U_COBERTURA, U_APORTES = 100, 0.80
U_FP_BAJO, U_EFICIENCIA = 30, 0.85
MAX_CENTRALES = 50


def leer_numero(mensaje, minimo, maximo, entero=False):
    while True:                                  # repite hasta dato valido
        texto = input(mensaje).replace(",", ".")
        try:
            valor = int(texto) if entero else float(texto)
        except ValueError:
            print("  ERROR: debe escribir un numero.")
            continue
        if minimo <= valor <= maximo:
            return valor
        print(f"  ERROR: el valor debe estar entre {minimo} y {maximo}.")


def leer_texto(mensaje):
    texto = input(mensaje).strip()
    while texto == "":
        print("  ERROR: el campo no puede quedar vacio.")
        texto = input(mensaje).strip()
    return texto.upper()


def calcular_central(c, dias, horas):
    pb = DENSIDAD * GRAVEDAD * c["qt"] * c["h"] * c["ef"] / 1e6   # MW
    if pb > c["pn"]:                             # RN-04
        pe, limitada = c["pn"], True
    else:
        pe, limitada = pb, False
    e = pe * horas / 1000                        # GWh
    fp = e / (c["pn"] * dias * 24 / 1000) * 100
    if limitada:                                 # RN-05
        estado = "LIMITADA"
    elif fp < U_FP_BAJO:
        estado = "SUBUTILIZADA"
    elif c["ef"] < U_EFICIENCIA:
        estado = "EFICIENCIA BAJA"
    else:
        estado = "ADECUADA"
    return pe, e, fp, estado


def registrar_generales():
    g = {"region": leer_texto("Region: ")}
    g["dias"] = leer_numero("Dias del mes: ", 28, 31, True)
    g["horas"] = leer_numero("Horas de operacion: ", 1, g["dias"] * 24, True)
```

```
    g["vu"] = leer_numero("Volumen util (Mm3): ", 1, 100000)
    g["vi"] = leer_numero("Volumen inicial (Mm3): ", 0, g["vu"])
    g["qmed"] = leer_numero("Caudal medio historico (m3/s): ", 1, 100000)
    g["dem"] = leer_numero("Demanda del mes (GWh): ", 1, 100000)
    return g


def registrar_central(centrales):
    if len(centrales) >= MAX_CENTRALES:        # RN-01
        print("  ERROR: maximo de centrales alcanzado.")
        return
    nombre = leer_texto("Nombre de la central: ")
    for c in centrales:                        # RN-06
        if c["nombre"] == nombre:
            print("  ERROR: esa central ya existe.")
            return
    centrales.append({"nombre": nombre,
        "qa": leer_numero("Caudal afluente (m3/s): ", 0, 10000),
        "qt": leer_numero("Caudal turbinado (m3/s): ", 0.1, 10000),
        "h": leer_numero("Altura neta (m): ", 1, 2000),
        "ef": leer_numero("Eficiencia (0.01-1): ", 0.01, 1),
        "pn": leer_numero("Potencia nominal (MW): ", 0.1, 2000)})


def consultar_central(centrales, g):
    nombre = leer_texto("Central a consultar: ")
    for c in centrales:
        if c["nombre"] == nombre:
            pe, e, fp, estado = calcular_central(c, g["dias"], g["horas"])
            print(f"  Pe={pe:.2f} MW  E={e:.2f} GWh  FP={fp:.1f} %  {estado}")
            return
    print("  No existe una central con ese nombre.")


def generar_reporte(centrales, g):
    et = va = vt = qa = pinst = emax = 0
    critica, malas = "", 0
    for c in centrales:                        # acumula por central
        pe, e, fp, estado = calcular_central(c, g["dias"], g["horas"])
        print(f"  {c['nombre']:<6} {pe:8.2f} MW {e:8.2f} GWh {fp:5.1f} % {estado}")
        et += e
        pinst += c["pn"]
        qa += c["qa"]
        va += c["qa"] * 86400 * g["dias"] / 1e6
        vt += c["qt"] * 3600 * g["horas"] / 1e6
        if e > emax:
            emax, critica = e, c["nombre"]
        if estado != "ADECUADA":
            malas += 1
    vf = g["vi"] + va - vt                     # balance del embalse
    vertido = max(0, vf - g["vu"])
    vf = min(max(vf, 0), g["vu"])
    nivel = vf / g["vu"] * 100
    ia = qa / g["qmed"]
    fp_tot = et / (pinst * g["dias"] * 24 / 1000) * 100
    cob = et / g["dem"] * 100
    deficit = g["dem"] - et if et < g["dem"] else 0
    autonomia = vf / ((vt / g["dias"]) * (g["dem"] / et))   # corregida
    nino = ia < U_APORTES                      # RN-10
    if nivel > U_NORMAL and cob >= U_COBERTURA:   # RN-07
        riesgo = "NORMAL"
    elif nivel > U_SEGUIM:
        riesgo = "SEGUIMIENTO"
    elif nivel > U_ALERTA:
        riesgo = "ALERTA"
    else:
        riesgo = "CRITICO"
    if riesgo == "CRITICO" or (deficit > 0 and nino):   # RN-08
        alerta = "ACTIVAR RESPALDO TERMICO Y GESTION DE DEMANDA"
    elif deficit > 0 or nino:
        alerta = "VIGILANCIA: PREPARAR RESPALDO"
    else:
        alerta = "OPERACION NORMAL"
```

```
    print(f"Energia {et:.2f} GWh | Nivel {nivel:.1f} % | Vertido {vertido:.1f} Mm3")
    print(f"IA {ia:.3f} | FP {fp_tot:.1f} % | Cobertura {cob:.1f} % | Deficit {deficit:.2f} GWh")
    print(f"Autonomia {autonomia:.1f} dias | Mayor aporte {critica} | Con problema {malas}")
    print(f"Riesgo: {riesgo} | Alerta: {alerta}")


generales, centrales, opcion = None, [], 0
while opcion != 5:                              # menu principal
    print("\n1.Datos generales 2.Registrar 3.Consultar 4.Reporte 5.Salir")
    opcion = leer_numero("Opcion: ", 1, 5, True)
    if opcion == 1:
        generales = registrar_generales()
    elif opcion == 2:
        registrar_central(centrales)
    elif opcion in (3, 4):
        if generales is None or len(centrales) == 0:   # RN-09
            print("  Registre primero los datos generales y una central.")
        elif opcion == 3:
            consultar_central(centrales, generales)
        else:
            generar_reporte(centrales, generales)
print("Fin del programa.")
```

Nota. Elaboración propia.

Pruebas de escritorio

La prueba de escritorio consiste en seguir el programa línea por línea con datos conocidos, habilidad necesaria para escribir programas correctos (Lister et al., 2004). Se usaron tres centrales didácticas de la Fase 1 (CH-1: H = 175 m, η = 0,90, P_n = 290 MW; CH-2: 150 m, 0,88, 165 MW; CH-3: 120 m, 0,86, 60 MW), con 30 días, volumen útil de 2 500 Mm³, caudal medio de 610 m³/s y demanda de 240 GWh. Los escenarios 1 a 4 aplican caudales de 100 %, 62 %, 38 % y 28 % de la condición base; el escenario 6 repite el 1 con demanda de 300 GWh y η de CH-3 igual a 0,75. Todas las salidas se obtuvieron ejecutando el programa.

Tabla 8

Matriz de pruebas de validación de entradas

Caso	Entrada	Regla	Salida esperada	Resultado
V-01	Opción 4 sin datos	RN-09	Pide registrar datos	Aprobado
V-02	Opción «abc»	Validación	Error: debe ser número	Aprobado
V-03	Opción 7	Validación	Error: entre 1 y 5	Aprobado
V-04	Región vacía	Validación	Error: campo vacío	Aprobado
V-05	Días = 32	RN-02	Error: entre 28 y 31	Aprobado
V-06	Horas = 800 con 30 días	RN-02	Error: entre 1 y 720	Aprobado
V-07	Vol. inicial 3 000 > útil 2 500	RN-03	Error: entre 0 y 2 500	Aprobado
V-08	Reporte sin centrales	RN-09	Pide registrar una central	Aprobado
V-09	Caudal = «x»	Validación	Error: debe ser número	Aprobado
V-10	Eficiencia = 1,5	Validación	Error: entre 0,01 y 1	Aprobado
V-11	Nombre «ch-1» ya registrado	RN-06	Error: central ya existe	Aprobado
V-12	Consultar «CH-9»	—	No existe la central	Aprobado
V-13	Consultar «ch-1» (Esc. 1)	RN-04, RN-05	290,00 MW; 145,00 GWh; 69,4 %; LIMITADA	Aprobado
V-14	Central número 51	RN-01	Error: máximo alcanzado	Aprobado

Nota. Elaboración propia.

Tabla 9

Matriz de resultados del reporte por escenario

Salida	Esc. 1	Esc. 2	Esc. 3	Esc. 4	Esc. 6
Condición	Neutro	Niño moderado	Niño extremo	Niño extremo, embalse bajo	Neutro, más demanda
Energía (GWh)	255,34	171,85	113,43	89,55	251,78
Nivel del embalse (%)	100,0	71,3	40,2	23,6	100,0
Vertido (Mm^3)	138,1	0,0	0,0	0,0	138,1
Índice de aportes (p. u.)	1,000	0,620	0,380	0,280	1,000
Factor de planta (%)	68,9	46,3	30,6	24,2	67,9
Cobertura (%)	106,4	71,6	47,3	37,3	83,9
Déficit (GWh)	0,00	68,15	126,57	150,45	48,22
Autonomía Fase 1 (días)	108,2	119,7	102,3	76,0	—
Autonomía corregida (días)	115,1	85,7	48,4	28,4	90,8
Centrales con problema	2 de 3	0 de 3	1 de 3	3 de 3	3 de 3
Riesgo	NORMAL	SEGUIMIENTO	ALERTA	CRÍTICO	SEGUIMIENTO
Alerta	NORMAL	ACTIVAR	ACTIVAR	ACTIVAR	VIGILANCIA
¿Coincide con lo esperado?	Sí	Sí	Sí	Sí	Sí

Nota. El escenario 5 de la Fase 1 (dato inválido) quedó cubierto por la Tabla 8. ACTIVAR abrevia «ACTIVAR RESPALDO TÉRMICO Y GESTIÓN DE DEMANDA». Elaboración propia.

Tabla 10

Prueba de escritorio del ciclo for del reporte (Escenario 6)

Iteración	Central	P_e (MW)	E (GWh)	FP (%)	Estado	et acumulada	malas
1	CH-1	290,00	145,00	69,4	LIMITADA	145,00	1
2	CH-2	165,00	82,50	69,4	LIMITADA	227,50	2
3	CH-3	48,56	24,28	56,2	EFICIENCIA BAJA	251,78	3

Nota. Elaboración propia.

Análisis de resultados

El programa reproduce los valores calculados manualmente en la Fase 1, lo que confirma que la traducción del pseudocódigo a Python es correcta. Con la corrección, la autonomía disminuye a medida que El Niño se intensifica (de 115,1 a 28,4 días). El estado por central permite identificar instalaciones deficientes: vertimiento en condición normal, subutilización en El Niño extremo y eficiencia baja en CH-3 (escenario 6). Las catorce pruebas de validación fueron aprobadas.

Conclusiones

En la Fase 2, el ciclo while controla el menú y la validación de datos, el ciclo for permite registrar, consultar y generar el reporte, y las condicionales aplican diez reglas de negocio verificadas con matrices de decisiones y pruebas de escritorio. Además de clasificar el riesgo, el programa señala qué central presenta problemas, lo que responde al eje problematizador.

Referencias

Cramton, P., y Stoft, S. (2007). Colombia firm energy market. En *Proceedings of the 40th Annual Hawaii International Conference on System Sciences (HICSS'07)* (p. 124). IEEE. https://doi.org/10.1109/HICSS.2007.133

Ehsan, H., Rehmat, A. P., y Cardella, M. E. (2020). Computational thinking embedded in engineering design: Capturing computational thinking of children in an informal engineering design activity. *International Journal of Technology and Design Education, 31*(3), 441–464. https://doi.org/10.1007/s10798-020-09562-5

Fangohr, H. (2004). A comparison of C, MATLAB, and Python as teaching languages in engineering. En M. Bubak, G. D. van Albada, P. M. A. Sloot y J. Dongarra (Eds.), *Computational Science – ICCS 2004* (Lecture Notes in Computer Science, Vol. 3039, pp. 1210–1217). Springer. https://doi.org/10.1007/978-3-540-25944-2_157

Glorfeld, L. W., y Palko, J. (1988). A comparison of novice algorithm composition performance using flowcharting or pseudocode as the design tool. *Journal of Research on Computing in Education, 21*(1), 82–96. https://doi.org/10.1080/08886504.1988.10781862

Gómez-Dueñas, S., Bateman, A., y Santos Granados, G. R. (2024). Untangling the implications of climate-forcing and human-induced drivers in streamflow variability: The Magdalena River, Colombia. *Hydrological Sciences Journal, 69*(8), 1046–1059. https://doi.org/10.1080/02626667.2024.2352517

Hoseinzadeh, T., Shourian, M., y Yazdi, J. (2020). Optimum design and operation of a hydropower reservoir considering uncertainty of inflow. *Journal of Hydroinformatics, 22*(6), 1452–1467. https://doi.org/10.2166/hydro.2020.044

Kunkle, W. M., y Allen, R. B. (2016). The impact of different teaching approaches and languages on student learning of introductory programming concepts. *ACM Transactions on Computing Education, 16*(1), 1–26. https://doi.org/10.1145/2785807

Lister, R., Adams, E. S., Fitzgerald, S., Fone, W., Hamer, J., Lindholm, M., McCartney, R., Moström, J. E., Sanders, K., Seppälä, O., Simon, B., y Thomas, L. (2004). A multi-national study of reading and tracing skills in novice programmers. *ACM SIGCSE Bulletin, 36*(4), 119–150. https://doi.org/10.1145/1041624.1041673

Medina, S., y Moreno, J. (2007). Risk evaluation in Colombian electricity market using fuzzy logic. *Energy Economics, 29*(5), 999–1009. https://doi.org/10.1016/j.eneco.2006.02.008

Moreno Chuquen, R., y Cantillo Luna, S. A. (2020). Análisis del riesgo de la cobertura de energía firme en Colombia a mediano plazo. *Tecnura, 24*(64), 23–33. https://doi.org/10.14483/22487638.16487

Poveda, G., Jaramillo, A., Gil, M. M., Quiceño, N., y Mantilla, R. I. (2001). Seasonality in ENSO-related precipitation, river discharges, soil moisture, and vegetation index in Colombia. *Water Resources Research, 37*(8), 2169–2178. https://doi.org/10.1029/2000WR900395

Qian, Y., y Lehman, J. (2017). Students' misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education, 18*(1), 1–24. https://doi.org/10.1145/3077618

Restrepo-Trujillo, J., Moreno-Chuquen, R., Jiménez-García, F. N., Flores, W. C., y Chamorro, H. R. (2022). Scenario analysis of an electric power system in Colombia considering the El Niño phenomenon and the inclusion of renewable energies. *Energies, 15*(18), Artículo 6690. https://doi.org/10.3390/en15186690

Sritram, P., y Suntivarakorn, R. (2017). Comparative study of small hydropower turbine efficiency at low head water. *Energy Procedia, 138*, 646–650. https://doi.org/10.1016/j.egypro.2017.10.181

Tayyeh, H. K., y Mohammed, R. (2024). Vulnerability and resilience of hydropower generation under climate change scenarios: Haditha dam reservoir case study. *Applied Energy, 366*, Artículo 123308. https://doi.org/10.1016/j.apenergy.2024.123308